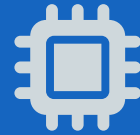


MODELAGEM COMPUTACIONAL



Máquinas de Turing - Parte 2

Prof. Dr. José Jairo de S. e Silva

Aula 08

Máquinas de Turing - Parte 2

OBJETIVOS DA AULA

🎯 Objetivos

Ao final desta aula, você deverá ser capaz de:

1. Enunciar a **Tese de Church-Turing** e entender por que ela é uma tese, e não um teorema.
2. Reconhecer que **variantes da MT** (multi-fita, não determinística, fita semi-infinita) têm o **mesmo poder** da MT padrão.
3. Codificar uma MT como uma string $\langle M \rangle$ e justificar por que essa codificação é fundamental.
4. Descrever a **Máquina de Turing Universal U** e o que significa "MT que executa outras MTs".
5. Definir a linguagem A_{TM} (Problema da Aceitação) e mostrar que ela é **RE**.
6. Compreender o argumento de **diagonalização** que prova que A_{TM} não é Recursiva.
7. Reconhecer o **Problema da Parada** ($HALT_{TM}$) como indecidível e suas consequências práticas.
8. Identificar linguagens que estão **fora de RE** e fechar a hierarquia: Regulares $\subsetneq$ LLCs $\subsetneq$ **R** $\subsetneq$ **RE** $\subsetneq$ Todas.

1. TESE DE CHURCH-TURING

🕒 Recapitulando a Aula 7 (parte 1): o modelo

Na aula anterior, vimos que a **Máquina de Turing** (abreviada **MT**) é um modelo de computação composto por uma **fita** infinita, um **cabeçote** que lê e escreve símbolos, e um **controle finito** (conjunto de estados).

A MT é definida por uma **7-tupla**:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F),$$

onde Q são os estados, Σ o alfabeto da entrada, Γ o alfabeto da fita (com símbolos extras como o branco B), δ a função de transição, q_0 o estado inicial e F os estados de aceitação.

A função δ tem a forma "estado atual + símbolo lido $\rightarrow$ novo estado + símbolo escrito + movimento". A MT aceita uma string w quando atinge um estado de F .

A cada passo, a MT pode: aceitar, rejeitar (parada sem aceitar), ou continuar rodando. Pode também entrar em **loop infinito**.

⌚ Recapitulando a Aula 7 (parte 2): as classes

A Aula 7 introduziu duas classes centrais de linguagens:

- **R** (linguagens **Recursivas**, também chamadas **Decidíveis**): aquelas que têm uma MT que **sempre para** (com aceita ou rejeita).
- **RE** (linguagens **Recursivamente Enumeráveis**): aquelas que têm uma MT que **reconhece** (aceita quando $w \in L$, mas pode rodar para sempre se $w \notin L$).

A relação entre as classes vistas até agora:

$$\text{Regulares} \subsetneq \text{LLCs} \subsetneq \mathbf{R} \subseteq \mathbf{RE}.$$

Construímos MTs para $\{0^n 1^n\}$, $\{w\#w\}$ e $\{a^n b^n c^n\}$: três linguagens **não regulares**, sendo duas delas também **não livres de contexto**.

A questão desta aula é: **a MT é o último modelo que precisamos?** Existe alguma máquina mais poderosa? Quanto a MT consegue, de fato, computar?

A Tese de Church-Turing

Na década de 1930, **antes mesmo de existirem computadores eletrônicos**, vários matemáticos propuseram formalismos diferentes para o conceito intuitivo de **algoritmo**:

- **Alonzo Church** (1936): cálculo lambda (λ -cálculo).
- **Alan Turing** (1936): Máquina de Turing.
- **Stephen Kleene** (1936): funções recursivas.
- **Emil Post** (1936): sistemas canônicos.
- **Kurt Gödel** e **Jacques Herbrand** (1934): funções recursivas gerais.

O **resultado surpreendente**: todos esses formalismos provaram ser **equivalentes** entre si. Cada um define a mesma classe de funções computáveis.

Tese de Church-Turing. Toda função intuitivamente computável pode ser computada por uma Máquina de Turing.

Equivalentemente: *o conjunto das funções computáveis por algoritmo é exatamente o conjunto das funções computáveis por MT.*

❓ Por que é uma "tese" e não um "teorema"?

Um **teorema** é uma afirmação matemática que pode ser provada a partir de definições e axiomas. A Tese de Church-Turing **não é** um teorema. Por quê?

Porque ela conecta dois conceitos de naturezas diferentes:

- **Lado esquerdo:** "função intuitivamente computável" - um conceito **informal**, baseado na ideia humana de "seguir um procedimento".
- **Lado direito:** "função computável por MT" - um conceito **formal**, com definição matemática precisa.

Não se pode provar uma equivalência entre um conceito informal e um conceito formal. O máximo que se pode fazer é **acumular evidências** de que os dois coincidem. E essas evidências são esmagadoras:

1. Toda tentativa séria de formalizar "computação" deu na mesma classe de funções.
2. Toda linguagem de programação real (Python, C, Lisp, Prolog, Rust) é equivalente à MT em poder.
3. Em quase 90 anos, ninguém propôs um modelo que computasse algo **fora** da classe de MT.

Em palavras simples: a Tese de Church-Turing diz "se você sabe escrever, em uma receita finita, como resolver um problema, então existe uma MT que resolve". É um princípio fundador da Ciência da Computação.

= MT é equivalente a sua linguagem favorita

A Tese de Church-Turing tem uma consequência prática direta:

Modelo de computação	Mesmo poder da MT?
Cálculo lambda de Church	sim
Funções recursivas de Kleene	sim
Máquinas de registradores (RAM)	sim
Autômatos celulares (ex. Regra 110, Game of Life)	sim
Python, C, Java, Rust, Go, Haskell, ...	sim
Sistema Lindenmayer (L-systems) sob certas condições	sim
Redes neurais artificiais com recorrência	sim
Computação quântica (em poder, não em eficiência)	sim

Mensagem central: qualquer dispositivo capaz de computar pode ser simulado por uma MT, e vice-versa. Portanto, ao estudar o que uma MT pode ou não pode fazer, estamos estudando os limites da computação em geral.

⚠ Implicação imediata

Se a Tese de Church-Turing é verdadeira (e é tratada como tal pela comunidade), então:

- Qualquer programa em Python pode ser convertido em uma MT;
- qualquer MT pode ser convertida em um programa Python;
- se algo é **indecidível** para uma MT, é **indecidível** para qualquer linguagem de programação.

O que significa "indecidível"? Uma linguagem (ou um problema) é **indecidível** quando **não existe** Máquina de Turing que sempre pare devolvendo a resposta correta. Pela Tese de Church-Turing, equivale a dizer: **não existe programa** (em nenhuma linguagem) que sempre termine e responda corretamente para toda entrada.

Os dois resultados centrais desta aula são indecidibilidades: o **Problema da Aceitação** (Seção 5) e o **Problema da Parada** (Seção 6). Veremos cada um deles em detalhe.

? Prévia: os dois problemas indecidíveis

Problema da Aceitação (veremos formalmente na Seção 5):

■ Dada uma MT M qualquer e uma entrada w , decidir se M aceita w .

Em código: existe uma função `aceita(programa, entrada)` que sempre devolve `True` ou `False`, corretamente, em tempo finito?

Problema da Parada (veremos formalmente na Seção 6):

■ Dada uma MT M qualquer e uma entrada w , decidir se M **termina** sobre w (sem ficar em loop).

Em código: existe uma função `para(programa, entrada)` com a mesma garantia?

Ambos parecem perguntas razoáveis. Ambos são **impossíveis** de resolver no caso geral, em qualquer linguagem de programação, qualquer hardware, qualquer modelo de computação. Não é limitação técnica; é limitação **lógica**.

2. VARIANTES DA MT SÃO EQUIVALENTES

MTs multi-fita

Uma **MT multi-fita** tem k fitas, cada uma com seu próprio cabeçote independente. A função de transição é generalizada para

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k.$$

A cada passo, a MT lê os k símbolos sob os k cabeçotes, decide com base nessa k -tupla, escreve novos k símbolos (um em cada fita) e move os k cabeçotes independentemente.

Convenções de inicialização:

- A entrada w é colocada na **fita 1**.
- As demais fitas começam **completamente em branco**.
- Todos os cabeçotes começam na primeira célula da sua respectiva fita.

Por que isso é útil? Programar uma MT com várias fitas é muito mais natural. Por exemplo, copiar uma string de uma fita para outra é trivial em uma MT de 2 fitas; em uma de fita única, exige marcar e voltar várias vezes.

🕒 Exemplo concreto: MT de 2 fitas para $\{w\#w\}$

Lembre da MT $M_{\#}$ da Aula 7: ela decidia $\{w\#w\}$ em uma única fita, com 5 estados auxiliares e marcadores X, Y . Cada comparação exigia ir e voltar pela fita inteira (custo quadrático).

Com 2 fitas, o algoritmo fica direto:

Fita 1 (entrada, somente leitura)	Fita 2 (rascunho)
$w\#w$ (cabeçote no início)	branco (cabeçote no início)

Algoritmo:

1. **Etapa de cópia.** Enquanto a fita 1 não atinge $\#$: leia o símbolo da fita 1, escreva-o na fita 2, mova ambos os cabeçotes para a direita.
 2. **Etapa de reposicionamento.** Ao chegar em $\#$ na fita 1: avance o cabeçote da fita 1 uma posição (passando o $\#$). Volte o cabeçote da fita 2 para o início.
 3. **Etapa de comparação.** Compare símbolo a símbolo: leia o símbolo da fita 1 e o da fita 2; se forem iguais, avance os dois cabeçotes; se forem diferentes, **rejeite**.
 4. **Etapa final.** Quando a fita 1 atinge o branco: se a fita 2 também atinge o branco, **aceite**; caso contrário, rejeite.
- Em palavras simples:** a fita 2 é usada como uma "fita de cópia". A primeira metade w é guardada lá, sem precisar marcar nem desmarcar nada. Depois, a comparação é uma simples leitura simultânea das duas fitas, da esquerda para a direita.

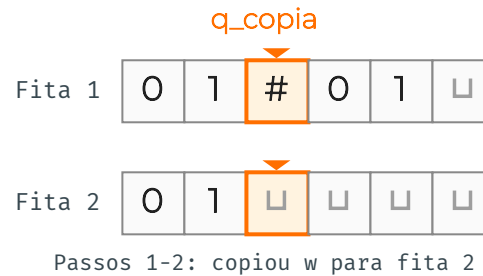
▶ Execução: 2 fitas sobre $w = 01\#01$ (parte 1)

A célula em **laranja** indica o cabeçote daquela fita. O estado da MT é mostrado em laranja no topo.

Estado inicial (q_{copia}): fita 1 contém a entrada; fita 2 está em branco.

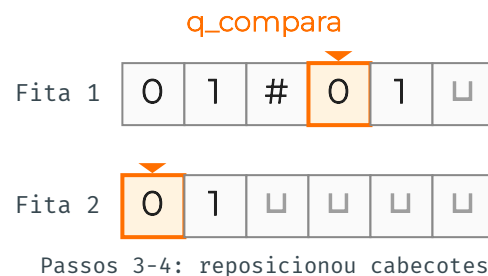


Após a etapa 1 (cópia: 2 passos). A fita 1 chegou em #; a fita 2 já contém a cópia $w = 01$.



► Execução: 2 fitas sobre $w = 01\#01$ (parte 2)

Após a etapa 2 (reposicionamento: 2 passos). Cabeçote 1 passou o #; cabeçote 2 voltou para o início.



Após a etapa 3 (comparação: 2 passos, símbolo a símbolo). Compara 0 com 0, depois 1 com 1. Ambos coincidem.



Total: 6 passos. Compare com $M_{\#}$ de fita única da Aula 7, que para $01\#01$ usaria mais de 20 passos.

🗂️ Outro exemplo: 3 fitas para $\{a^n b^n c^n\}$

A MT de fita única da Aula 7 (M_{abc}) precisava de marcadores (x, y, z) e múltiplas passadas. Com 3 fitas o algoritmo fica trivial:

Fita	Papel
1	entrada (somente leitura)
2	contador unário dos <i>a</i> s lidos
3	contador unário dos <i>b</i> s lidos

Algoritmo:

- 1. **Fase a:** enquanto a fita 1 vê *a*, escreva 1 na fita 2 e avance ambos os cabeçotes.
 - 2. **Fase b:** enquanto a fita 1 vê *b*, escreva 1 na fita 3 e avance ambos os cabeçotes.
 - 3. **Fase c:** ao ver *c* na fita 1, volte os cabeçotes 2 e 3 para o início. Para cada *c* na fita 1, **apague um 1** das fitas 2 e 3 (avance os três cabeçotes).
 - 4. **Final:** se a fita 1 atinge o branco e as fitas 2 e 3 também, **aceite**. Caso contrário, rejeite.
- Compare com a versão de fita única:** lá, M_{abc} percorria a fita inteira *n* vezes (uma por trio marcado), gastando $O(n^2)$ passos. Aqui, percorremos a entrada **uma única vez**, gastando $O(n)$ passos. Os contadores nas fitas 2 e 3 substituem os marcadores e as travessias.

MT de 4 fitas para $\{a^n b^n c^n\}$ no simulador

Versão para turingmachinesimulator.com com 4 fitas (entrada + um contador por símbolo). Sintaxe: duas linhas por transição.

```
name: anbncn_4fitas
init: q0
accept: qAccept
// le a's, grava fita 2
q0,a,_,_,_
q0,a,a,_,_,>,>,-,-
// 1o b: vai fase B
q0,b,_,_,_
q1,b,_,b,_,>,-,>,-
// n=0
q0,_,_,_,_
qAccept,_,_,_,_,-, -, -, -
// le b's, grava fita 3
q1,b,_,_,_
q1,b,_,b,_,>,-,>,-
// 1o c: vai fase C
q1,c,_,_,_
q2,c,_,_,c,>,-,-,>

// le c's, grava fita 4
q2,c,_,_,_
q2,c,_,_,c,>,-,-,>
// fim: volta cabecotes 2,3,4
q2,_,_,_,_
qConf,_,_,_,_,-,<,<,<
// confere a,b,c simultaneos
qConf,_,a,b,c
qConf,_,a,b,c,-,<,<,<
// tres contadores zeram juntos
qConf,_,_,_,_
qAccept,_,_,_,_,-, -, -, -
```

Teste: **abc**, **aabbcc**, **aaabbbccc** aceitam; **aabc**, **abbcc** rejeitam. Custo: $O(n)$.

 MT de 2 fitas para $\{a^n b^n c^n\}$ no simulador

Versão com 2 fitas: marca 1 na fita 2 para cada a , rebobina e consome um 1 por b , rebobina de novo e consome um 1 por c .

```

name: anbncn_2fitas
init: q0
accept: qAccept

// FASE A: marca 1 por a
q0,a,_
q0,a,1,>,>

// 1o b: rebobina fita 2
q0,b,_
qRew1,b,-,<

// n=0
q0,-,_
qAccept,-,-,-

// REBOBINA 1
qRew1,b,1
qRew1,b,1,-,<
qRew1,b,_
q1,b,-,-,>

// FASE B: cada b consome um 1
q1,b,1
q1,b,1,>,>

// 1o c: rebobina de novo
q1,c,_
qRew2,c,-,-,<

// REBOBINA 2
qRew2,c,1
qRew2,c,1,-,<
qRew2,c,_
q2,c,-,-,>

// FASE C: cada c consome um 1
q2,c,1
q2,c,1,>,>

// aceita
q2,-,_
qAccept,-,-,-
    
```

Os "buracos" não definidos em δ (ex: $q_1, b, _$ ou $q_1, c, 1$) atuam como rejeição.

 MT de 1 fita para $\{a^n b^n c^n\}$ no simulador

Versão de **fita única**: marca um a como **X**, um b como **Y**, um c como **Z**; volta ao início; repete. Custo: $O(n^2)$.

```
name: anbncn_1fita
init: q0
accept: qAccept

// FASE A: acha 'a'
q0,a
q1,X,>
q0,Y
q0,Y,>
q0,_
qAccept,_,-

// FASE B: acha 'b'
q1,a
q1,a,>
q1,Y
q1,Y,>
q1,b
q2,Y,>

// FASE C: acha 'c'
q2,b
q2,b,>
q2,Z
q2,Z,>
q2,c
q3,Z,<

// RETORNO: volta ao inicio
q3,a
q3,a,<
q3,b
q3,b,<
q3,Y
q3,Y,<
q3,Z
q3,Z,<
q3,X
q0,X,>

// VALIDACAO
q0,Z
q4,Z,>
q4,Z
q4,Z,>
q4,_
qAccept,_,-
```

Compare as três versões: 4 fitas $O(n)$, 2 fitas $O(n)$ com rebobinagens, 1 fita $O(n^2)$. Mesma linguagem, eficiências distintas.

↔ MT de 2 fitas para $\{w\#w\}$ no simulador

Versão com 2 fitas: copia tudo antes do $\#$ para a fita 2; rebobina a fita 2; compara os dois lados símbolo a símbolo. Custo: $O(n)$.

```

name: wcw_2fitas
init: q0
accept: qAccept

// FASE A: copia esquerda
// para fita 2
q0,0,_
q0,0,0,>,>
q0,1,_
q0,1,1,>,>

// chegou em #: começa
// rebobina da fita 2
q0,#,_
qRew,#,_,>,<

// REBOBINA fita 2
// (head1 fica parado)
qRew,0,0
qRew,0,0,-,<
qRew,0,1
qRew,0,1,-,<
qRew,1,0
qRew,1,0,-,<
qRew,1,1
qRew,1,1,-,<

// fita 2 chegou em _:
// vai para fase B
qRew,0,_
q1,0,-,-,>
qRew,1,_
q1,1,-,-,>

// FASE B: compara simbolo
// a simbolo
q1,0,0
q1,0,0,>,>
q1,1,1
q1,1,1,>,>

// terminam juntos: aceita
q1,-,-
qAccept,-,-,-
    
```

Teste com 01#01, 0110#0110, # (aceitam) e 01#10, 00#01 (rejeitam). Buracos em δ (ex: $q_1, 0, 1$) rejeitam por mismatch.

↔ MT de 1 fita para $\{w\#w\}$ no simulador (parte 1)

Versão de **fita única** (a $M_{\#}$ da Aula 7). A cada rodada marca um símbolo à esquerda (X para 0, Y para 1) e o correspondente à direita. Custo: $O(n^2)$. Esta parte cobre o estado inicial **i** e a travessia até o **#** (**m0** e **m1**).

```

name: wcw_1fita
init: i
accept: A

// FASE A: marca esquerda
i,0
m0,X,>
i,1
m1,Y,>
i,#
v_dir,#,>

// m0: anda ate # (caso 0)
m0,0
m0,0,>
m0,1
m0,1,>
m0,X
m0,X,>
m0,Y
m0,Y,>
m0,#
n0,#,>

// m1: anda ate # (caso 1)
m1,0
m1,0,>
m1,1
m1,1,>
m1,X
m1,X,>
m1,Y
m1,Y,>
m1,#
n1,#,>

```

Quando **i** lê **#** já no início da rodada, toda a esquerda já está marcada $\Rightarrow$ vai para **v_dir** (verificação à direita).

↔ MT de 1 fita para $\{w\#w\}$ no simulador (parte 2)

Continuação: estados que **casam o símbolo à direita** (**n0**, **n1**), **voltam** ao próximo não marcado da esquerda (**b**, **c**) e **verificam** que sobraram só marcadores (**v_dir**).

```
// n0: acha 0 nao marcado direita
n0,X
n0,X,>
n0,Y
n0,Y,>
n0,0
b,X,<

// n1: acha 1 nao marcado direita
n1,X
n1,X,>
n1,Y
n1,Y,>
n1,1
b,Y,<

// b: volta direita ate o #
b,X
b,X,<
b,Y
b,Y,<
b,0
b,0,<
b,1
b,1,<
b,#
c,#,<
```

```
// c: anda esquerda ate o ultimo
// X/Y (ou inicio da fita) e ja
// pula uma celula para a direita
c,0
c,0,<
c,1
c,1,<
c,X
i,X,>
c,Y
i,Y,>
c,-
i,-,>

// VERIFICACAO direita: so X/Y
v_dir,X
v_dir,X,>
v_dir,Y
v_dir,Y,>
v_dir,-
A,-,-
```

▮ Cole parte 1 + parte 2 no simulador e teste com **01#01**, **001#001**, **0110#0110** (aceitam) versus **01#10**, **0#1**, **00#01** (rejeitam).

🕒 O que significa esse n ? E como o número de fitas impacta?

O n é o tamanho da entrada, em símbolos. Para $w\#w$ com $|w| = 5$, temos $n = 11$ (cinco símbolos + o $\#$ + cinco símbolos). Para $a^n b^n c^n$ com $n = 4$, temos uma string de tamanho $3n = 12$.

O número de fitas, k , NÃO entra no n . Ele é uma constante do projeto da MT (geralmente $k = 1, 2, 3, \dots$), escolhida pelo projetista e fixa para toda execução.

Como k afeta o tempo de execução?

Linguagem	1 fita	2+ fitas
$\{w\#w\}$	$O(n^2)$	$O(n)$
$\{a^n b^n c^n\}$	$O(n^2)$	$O(n)$
Palíndromos $\{w w^R\}$	$\Theta(n^2)$	$\Theta(n)$

A intuição: a fita única obriga o cabeçote a percorrer toda a entrada **uma vez para cada par marcado**, gastando n idas-e-voltas, totalizando $n \cdot n = n^2$ movimentos. Com fitas auxiliares, cada cabeçote anda **de forma independente**, então as comparações acontecem **em paralelo**: uma única travessia basta.

Em palavras simples: mais fitas $\neq$ mais poder computacional. A classe de linguagens reconhecíveis **não muda**. O que muda é o **número de passos**: na fita única, o cabeçote "desperdiça" tempo voltando para reler; com fitas extras, cada uma guarda o seu estado e nada precisa ser refeito.

✂ Teorema: multi-fita equivale a fita única

Teorema 5.4.1. Para toda MT M com k fitas, existe uma MT M' com **uma única fita** tal que $L(M') = L(M)$.

Esboço da simulação. A MT de fita única M' usa uma fita "ampliada" com $2k$ trilhas, separadas por símbolos especiais. Cada par de trilhas representa uma fita de M :

- Trilha $2i - 1$: armazena o **conteúdo** da i -ésima fita de M .
- Trilha $2i$: contém uma **marca** (●) apontando onde está o cabeçote da i -ésima fita.

Para simular **um passo** de M , a MT M' executa um procedimento:

1. **Varredura para a direita**, anotando no estado os k símbolos marcados.
2. Aplica δ de M (codificada nos estados de M').
3. **Varredura para a esquerda**, atualizando as k trilhas com novos símbolos e novas posições dos cabeçotes.

Custo: cada passo de M vira $O(\text{tamanho da fita})$ passos de M' . O **poder** é o mesmo, mas a **eficiência** muda.

🌀 Mais fitas significa "rodar mais rápido"?

Essa é a pergunta natural depois do teorema da equivalência. A resposta tem **duas camadas**.

- 1. **Em termos de PODER computacional: não muda nada.** A classe de linguagens reconhecidas é exatamente a mesma. Tudo que uma MT de 7 fitas decide, uma MT de 1 fita também decide. As duas reconhecem **RE**, e as duas que sempre param decidem **R**.
- 2. **Em termos de EFICIÊNCIA (número de passos): sim, é mais rápido.** Adicionar fitas costuma reduzir, polinomialmente, o número de passos necessários para reconhecer a mesma linguagem.

Linguagem	MT de 1 fita	MT de 2+ fitas
Palíndromos $\{w \mid w = w^R\}$	$O(n^2)$	$O(n)$
$\{w\#w\}$	$O(n^2)$	$O(n)$
$\{a^n b^n c^n\}$	$O(n^2)$	$O(n)$
Multiplicação de inteiros em binário	$O(n^2)$	$O(n \log n)$

Em palavras simples: as fitas extras funcionam como **memória scratch** (rascunho). Em vez de marcar e voltar dezenas de vezes, a MT mantém estruturas auxiliares em outras fitas. Como em um programa real, ter mais variáveis acelera mas não muda o que se pode calcular.

 Analogia com programação de verdade

Pense na diferença entre MT de 1 fita e MT de k fitas como a diferença entre dois estilos de programação:

MT de 1 fita	MT de k fitas
Tudo em uma única string, no mesmo lugar onde está a entrada.	Entrada em uma variável, scratch em outras variáveis separadas.
Para "lembrar" algo, é preciso marcar a fita e voltar nela.	Para "lembrar" algo, basta escrever em outra fita.
Reordenar dados exige $O(n)$ por operação.	Reordenar é gratuito (basta usar uma fita auxiliar).

Por isso programadores escrevem em alto nível e não em fita única. As linguagens modernas oferecem variáveis, listas, dicionários, etc., que correspondem informalmente a "fitas extras". O poder é o mesmo da fita única, mas a clareza e a eficiência são incomparáveis.

Consequência teórica. Para análises de complexidade (classes P, NP, PSPACE da Aula 10), a comunidade adota o modelo multi-fita, justamente porque ele captura melhor o "tempo real" gasto por algoritmos práticos.

Resumo: mais fitas $\Rightarrow$ não amplia o que se pode computar; **sim** acelera quase tudo; **sim** torna a programação muito mais clara.

✂ MTs não determinísticas (NMT)

Em uma **MT não determinística**, a função de transição δ devolve um **conjunto** de possibilidades em vez de uma única ação:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R, S\}).$$

A cada passo, a NMT pode escolher **uma entre várias** transições disponíveis. Aceita w se **existe pelo menos uma** sequência de escolhas que leva a um estado final.

Teorema 5.4.2. Para toda NMT N , existe uma MT determinística M tal que $L(M) = L(N)$.

Esboço da simulação. A MT determinística M usa **três fitas** (que por sua vez podem ser compactadas em uma):

- **Fita 1:** a entrada w (somente leitura).
- **Fita 2:** simulação do trabalho de N sobre w ao longo de **um** caminho não determinístico.
- **Fita 3:** ponteiro para o **próximo caminho** a explorar (numerado em ordem BFS).

M percorre todos os caminhos de N em **largura** (BFS), simulando cada um por um número fixo de passos. Se algum caminho aceita, M aceita.

Por que BFS e não DFS? Porque um caminho pode entrar em loop. DFS poderia ficar preso em um ramo infinito enquanto outro ramo de tamanho finito aceitaria. BFS garante que todo caminho aceitante de comprimento n é explorado em tempo finito.

 Outras variantes equivalentes

Várias outras variantes da MT foram propostas, e **todas** se mostraram equivalentes à MT padrão:

Variante	Diferença	Equivalente?
Fita semi-infinita (só à direita)	Não há células à esquerda da inicial	sim
Fita bidimensional (grade)	Cabeçote anda em 4 direções	sim
MT com várias cabeças, uma fita	<i>k</i> cabeçotes na mesma fita	sim
MT enumeradora	Imprime as strings de <i>L</i> em vez de aceitar	sim (mesma classe RE)
MT com pilha extra (autômato com 2 pilhas)	Pilhas em vez de fita	sim
MT com fila (autômato com fila)	Fila em vez de fita	sim

Conclusão. A MT padrão é **robusta**: não importa qual variante razoável você escolha, o conjunto de linguagens reconhecíveis é o mesmo. Isso reforça a Tese de Church-Turing.

3. CODIFICANDO UMA MT COMO UMA STRING

💡 A ideia em uma frase

| Codificar uma MT é transformar uma "máquina" em um arquivo de texto.

A partir do momento que a MT vira uma string, ela deixa de ser apenas uma ideia matemática e passa a ser **dado**: pode ser copiada, comparada, gravada em uma fita, mandada por e-mail, e o mais importante, **dada como input para outra MT**.

É exatamente esse o salto que vamos dar nas próximas seções. Antes de codificar, "MT" e "string" são objetos de mundos diferentes. Depois de codificar, **toda MT é uma string**, e podemos misturar tudo.

| Sem essa transformação, perguntas como *"esse programa termina sobre essa entrada?"* nem fazem sentido formal. Com ela, viram **perguntas sobre strings**, que outras MTs podem (ou não) responder.

🔗 Analogia: o programa Python que está no seu disco

Pense em um programa Python salvo no seu computador. Fisicamente, esse programa é só uma **sequência de bytes** em um arquivo, letras e símbolos. Você consegue:

- abrir o arquivo no editor;
- copiar para um pendrive;
- mandar por e-mail;
- comparar com outro arquivo;
- contar quantos bytes ele tem.

Nesse momento, o programa é **dado passivo**.

Quando você manda o interpretador Python rodar esse arquivo, ele **lê o texto** e **executa as instruções**. Aí o arquivo deixa de ser passivo e vira **comportamento**: cálculos, prints na tela, efeitos.

Note que o arquivo **continua sendo** uma string de bytes mesmo durante a execução. O que muda é que existe um **outro programa** (o interpretador) que sabe ler aquela string e agir conforme ela.

↔ Aplicando a analogia à Máquina de Turing

Uma Máquina de Turing começa do **lado abstrato**: ela é só uma definição matemática (a 7-tupla). Em si, é uma ideia em um papel, não um arquivo.

Codificar uma MT é dar a ela uma forma de arquivo. É escrever a tabela de transição inteira em uma única string em $\{0, 1\}^*$.

Programação real	Máquina de Turing
Programa Python (código-fonte como texto)	$\langle M \rangle$ (descrição da MT como string)
Interpretador Python	MT Universal U
Executar o programa = interpretador lê o código e age conforme ele	Simular $M = U$ lê $\langle M \rangle$ e age conforme ele
Dados em outro arquivo	A entrada w , codificada também
Chamada <code>python programa.py entrada.txt</code>	U rodando sobre $\langle M, w \rangle$

Resumo. Codificar = **transformar máquina em dado**, para que máquinas possam manipular outras máquinas. É o passo que conecta o universo abstrato das MTs ao universo concreto dos programas, e que viabiliza tudo que vem depois: MT Universal, Problema da Aceitação, Problema da Parada.

</> Por que codificar uma MT?

Até agora, a MT M era um **objeto matemático**: uma 7-tupla com conjuntos finitos. A entrada da MT era uma **string** $w \in \Sigma^*$.

Mas para falar de uma **MT que recebe outras MTs como entrada** (a MT Universal do próximo bloco, ou A_{TM}), precisamos **representar a própria MT como uma string**.

Notação. A string que codifica a MT M é escrita $\langle M \rangle$. Para um par de objetos (MT e entrada), escrevemos $\langle M, w \rangle$.

Critério para uma boa codificação. A codificação $\langle M \rangle$ deve:

- ser **finita** (sempre tem tamanho finito, mesmo que grande);
- conter **todos** os dados de M (estados, alfabetos, tabela de transição, estado inicial, símbolo branco, estados finais);
- permitir **decodificar** M a partir de $\langle M \rangle$ por um procedimento mecânico (uma outra MT).

Não importa o esquema exato: o importante é que ele exista. Diferentes esquemas levam a strings diferentes, mas todos são equivalentes do ponto de vista da computabilidade.

</> Um esquema simples de codificação

Suponha $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$. Numeramos os elementos:

- Estados: $q_1, q_2, q_3, \dots, q_{|Q|}$, com $q_1 = q_0$.
- Símbolos da fita: $X_1, X_2, X_3, \dots, X_{|\Gamma|}$, com $X_1 = B$ (branco).
- Movimentos: $D_1 = L, D_2 = R, D_3 = S$.

Codificação de uma transição $\delta(q_i, X_j) = (q_k, X_\ell, D_m)$:

$$0^i 1 0^j 1 0^k 1 0^\ell 1 0^m$$

(números unários separados por 1).

Codificação de M inteira: lista de todas as transições, separadas por 11.

Codificação de $\langle M, w \rangle$: $\langle M \rangle$, seguido de 111, seguido de w (interpretando os símbolos de w pelos seus índices).

Em palavras simples: $\langle M \rangle$ é só a tabela de transição de M escrita em binário, com convenções fixas para estados, símbolos e movimentos. Curto ou comprido, é apenas uma string em $\{0, 1\}^*$.

≡ Por que separadores 1, 11 e 111?

A codificação tem **três níveis** de informação aninhados, e cada nível usa um número diferente de 1s para se separar. A razão é simples: cada padrão de separador **não pode aparecer dentro** do conteúdo que ele separa.

Nível 1: dentro de uma transição, separador é 1.

Em $0^i 1 0^j 1 0^k 1 0^\ell 1 0^m$, cada bloco de 0s representa um número (estado, símbolo, movimento). Entre dois blocos sempre há **exatamente um 1**. Portanto, dentro de uma transição **nunca aparecem dois 1s consecutivos**.

Nível 2: entre transições, separador é 11.

Quando uma MT tem várias transições, juntamos suas codificações com **11** no meio:

$$\underbrace{0^{i_1} 1 0^{j_1} 1 0^{k_1} 1 0^{\ell_1} 1 0^{m_1}}_{\text{transição 1}} \quad 11 \quad \underbrace{0^{i_2} 1 0^{j_2} 1 \dots}_{\text{transição 2}}$$

Funciona porque **11 nunca aparece dentro** de uma transição (vimos acima que dentro só há **1** isolado). Logo, ao ler $\langle M \rangle$, todo **11** marca o fim de uma transição e o começo da próxima.

Nível 3: entre $\langle M \rangle$ e w , separador é 111.

Já dentro de $\langle M \rangle$ pode haver **11** (entre transições). Para separar a descrição da MT do dado de entrada, usamos **111**, que **nunca aparece dentro** de $\langle M \rangle$.

Resumo. Cada nível mais externo usa **um 1 a mais** que o anterior, garantindo unicidade na decodificação. É o mesmo princípio das **aspas triplas** em Python (`"""..."""`): quando o nível interno já usa um delimitador, o externo usa uma versão mais longa

👁 Exemplo visual dos três níveis

Imagine uma MT M com **duas** transições T_1 e T_2 , rodando sobre uma entrada w . A string $\langle M, w \rangle$ tem a estrutura:

$$\overbrace{0 \dots 0 \mathbf{1} 0 \dots 0 \mathbf{1} 0 \dots}^{T_1 \text{ (separadores 1)}} \mathbf{11} \overbrace{0 \dots 0 \mathbf{1} 0 \dots}^{T_2 \text{ (separadores 1)}} \mathbf{111} \underbrace{0 \dots 0}_{w \text{ codificada}}$$

$\langle M \rangle$ (transições separadas por 11)

Padrão observado	Significado
1 isolado	separador entre campos dentro de uma transição
11 (dois 1 s seguidos)	fim de uma transição, começo da próxima
111 (três 1 s seguidos)	fim de $\langle M \rangle$, começo de w
1111 ou mais	não ocorre em uma codificação válida

Ao **decodificar**, a outra MT (a Universal U) varre a string da esquerda para a direita procurando blocos de **1**s. O **comprimento** desse bloco (1, 2 ou 3) diz a ela em qual nível está: campo, transição ou separador entrada/MT.

🕒 Objetivo deste exemplo

Você acabou de ver as regras de codificação no abstrato. Agora vamos **fazer** uma codificação inteira, do começo ao fim, sobre a **menor MT possível**.

Por que isso importa? Três razões concretas:

1. **Verificar que dá certo.** As regras envolvem cinco campos, dois alfabetos, três tipos de separador. Aplicar isso em um caso pequeno mostra que tudo encaixa em uma string finita.
2. **Treinar a decodificação.** Em poucos slides vamos construir a **MT Universal U** : uma MT que recebe $\langle M, w \rangle$ na fita e simula M sobre w . Para entender o que U faz, precisamos ter feito a codificação manualmente ao menos uma vez.
3. **Fundamentar o que vem a seguir.** Resultados como " A_{TM} é indecidível" e o Problema da Parada falam sobre MTs como **dados**. Sem codificação, essas frases nem fazem sentido. O exemplo aqui é o ponto onde "MT como objeto matemático" vira "MT como string".

Em palavras simples: este exemplo é o "Hello, World" da codificação de MTs. Ele não é interessante por si — é interessante porque garante que o resto da aula está em terreno sólido.

📊 Exemplo (1/4): a MT que vamos codificar

Escolhemos a **menor MT interessante possível**: dois estados, uma única transição. O que ela faz: lê a entrada; se for exatamente a string **0**, aceita; qualquer outra coisa, rejeita.

Como 7-tupla:

$$M_0 = (\{q_1, q_2\}, \{0\}, \{0, B\}, \delta, q_1, B, \{q_2\}).$$

Componentes:

- $Q = \{q_1, q_2\}$: dois estados.
- $\Sigma = \{0\}$: a entrada só pode conter **0**.
- $\Gamma = \{0, B\}$: a fita contém **0** e o branco **B**.
- $q_0 = q_1$: começa em q_1 .
- $F = \{q_2\}$: aceita ao chegar em q_2 .

A função δ tem uma única regra: **estando em q_1 lendo 0, escreve 0, move à direita e vai para q_2** :

$$\delta(q_1, 0) = (q_2, 0, R).$$

Em qualquer outro par (q, X) , δ é indefinida $\Rightarrow$ a MT rejeita.

Exemplo (2/4): atribuindo índices numéricos

Para escrever M_0 como uma string binária, precisamos transformar **estados**, **símbolos** e **movimentos** em **números**. Adotamos uma numeração fixa:

Estados. Começa-se pelo inicial e segue na ordem em que aparecem:

$$q_1 \mapsto 1, \quad q_2 \mapsto 2.$$

Símbolos da fita. Convenção: X_1 é sempre o branco B ; os demais seguem na ordem em que aparecem.

$$X_1 = B \mapsto 1, \quad X_2 = 0 \mapsto 2.$$

Movimentos. Convenção fixa de três valores:

$$D_1 = L \mapsto 1, \quad D_2 = R \mapsto 2, \quad D_3 = S \mapsto 3.$$

Com isso, a transição $\delta(q_1, 0) = (q_2, 0, R)$ vira, em termos de **índices**:

$$\delta(q_1, X_2) = (q_2, X_2, D_2) \Leftrightarrow (i, j, k, \ell, m) = (1, 2, 2, 2, 2).$$

■ A próxima etapa apenas escreve esses cinco números em **unário** (com 0s) separados por 1s.

</> Exemplo (3/4): montando a string $\langle M_0 \rangle$

A regra geral para codificar uma transição é

$$\underbrace{0^i}_{\text{estado origem}} \ 1 \ \underbrace{0^j}_{\text{símbolo lido}} \ 1 \ \underbrace{0^k}_{\text{estado destino}} \ 1 \ \underbrace{0^\ell}_{\text{símbolo escrito}} \ 1 \ \underbrace{0^m}_{\text{movimento}} \ .$$

Cada número vira um bloco de 0s; cada 1 é separador. Para $(i, j, k, \ell, m) = (1, 2, 2, 2, 2)$:

Campo	Índice	Bloco unário
i (estado origem q_1)	1	0
j (símbolo lido $X_2 = 0$)	2	00
k (estado destino q_2)	2	00
ℓ (símbolo escrito)	2	00
m (movimento R)	2	00

Concatenando blocos com 1 entre eles:

$$\langle M_0 \rangle = 0 \ 1 \ 00 \ 1 \ 00 \ 1 \ 00 \ 1 \ 00 = 0100100100100$$

13 bits ao todo. Como M_0 tem só uma transição, essa é a codificação completa.

🔗 Exemplo (4/4): codificando o par $\langle M_0, w \rangle$

Para representar a MT **e** a entrada juntas, usamos o separador especial **111**:

$$\langle M, w \rangle = \underbrace{\langle M \rangle}_{\text{descrição da MT}} \underbrace{111}_{\text{separador}} \underbrace{w_{\text{codif.}}}_{\text{entrada codificada}} .$$

A entrada w também precisa ser codificada: cada símbolo X_j é escrito como 0^j .

Caso concreto: queremos codificar o par $\langle M_0, w \rangle$ com $w = 0$. Como 0 é o símbolo de índice $j = 2$, codificamos w como **00**.

Juntando tudo:

$$\langle M_0, 0 \rangle = \underbrace{0100100100100}_{\langle M_0 \rangle} \underbrace{111}_{\text{separador}} \underbrace{00}_{w=0} .$$

Em palavras simples: todos os dados de uma MT (tabela de transição) e sua entrada cabem em uma **única** string em $\{0, 1\}^*$. Isso é o que permite construir a **MT Universal** (próxima seção): uma MT que recebe a descrição de outra como dado de entrada.

Recapitulando o objetivo: começamos com M_0 , um objeto matemático abstrato (7-tupla). Terminamos com a string concreta **0100100100100 111 00**. A partir daqui, podemos usar essa string como **entrada** de outra MT — exatamente o que faz a MT Universal, e o que permite formular e resolver perguntas sobre o **comportamento de MTs** (próximas seções).

▼ Nem toda string é uma codificação válida

Considere o alfabeto $\{0, 1\}$. Existem strings em $\{0, 1\}^*$ que **não** são codificações de MT alguma. Por exemplo:

- A string vazia ϵ ;
- A string **0** (curta demais);
- Strings com formato malformatado, sem os **1**s separadores corretos.

Convenção útil. Quando lidamos com strings $x \in \{0, 1\}^*$ que **não** são codificações válidas, podemos:

- definir uma MT padrão "lixo" $M_{\perp}$ que rejeita imediatamente; e
- estipular que toda string mal formada x **é** $\langle M_{\perp} \rangle$ por convenção.

Assim, **toda** string em $\{0, 1\}^*$ corresponde a alguma MT (a maioria delas a $M_{\perp}$).

Essa convenção simplifica as definições de linguagens como A_{TM} , que serão dadas em termos de pares $\langle M, w \rangle$ sem precisar listar exceções a cada parágrafo.

4. MÁQUINA DE TURING UNIVERSAL

☰ A ideia da MT Universal

Toda MT padrão M é **especializada** em uma única tarefa. Por exemplo, a MT M_{01} da Aula 7 só sabe reconhecer $\{0^n 1^n\}$. Para reconhecer outra linguagem, é preciso uma **outra** MT.

A pergunta de Turing em 1936 foi:

Existe **uma única MT** U que, recebendo como entrada a descrição de qualquer outra MT M junto com uma string w , simule a execução de M sobre w ?

A resposta é **sim**. Essa MT é chamada de **Máquina de Turing Universal** (U).

Significado profundo. A existência de U é o **fundamento teórico** do conceito de **computador programável**. Antes de Turing, cada problema exigia uma máquina dedicada. A MT Universal mostrou que **uma única máquina** (com hardware fixo) pode executar **qualquer** algoritmo, desde que ele seja fornecido como dado.

Em palavras concretas: U é uma "MT que executa MTs". A relação U vs M é a mesma que existe entre um **interpretador Python** (U) e um **programa Python** (M).

⚙️ Como U funciona

A MT Universal U recebe na entrada $\langle M, w \rangle$ (a codificação de M seguida de w) e simula a execução de M sobre w .

Estrutura interna típica (com 3 fitas, equivalente à fita única):

- **Fita 1:** $\langle M \rangle$, a "tabela de transição" da MT simulada. Mantida fixa após a leitura.
- **Fita 2:** simula a fita de M . Inicialmente contém w . Depois, evolui passo a passo.
- **Fita 3:** estado atual de M (apenas o índice do estado, ex. 0^k para q_k).

A cada passo de simulação, U :

1. Lê o símbolo sob o cabeçote da fita 2 (o "símbolo sob o cabeçote de M ").
2. Lê o estado atual da fita 3.
3. **Procura na fita 1** a entrada (q, X) que corresponde a esse par e lê a saída (q', X', d) .
4. **Atualiza** a fita 2 (escreve X' e move o cabeçote) e a fita 3 (escreve q').
5. Verifica se q' é final. Se for, aceita.

Em palavras simples: U "lê o manual" ($\langle M \rangle$) e segue as instruções passo a passo, atualizando a fita simulada. É um **interpretador** universal.

► Execução de U sobre $\langle M_0, 0 \rangle$

Vamos rastrear U simulando a MT M_0 do exemplo anterior (aceita exatamente a string 0).

Configuração inicial das 3 fitas de U :

Fita 1 (programa): 0100100100100 (= $\langle M_0 \rangle$)
 Fita 2 (dado): $\underset{\wedge}{0}$ - - - - - (= entrada $w = 0$)
 Fita 3 (estado): $\underset{\wedge}{0}$ - - - - - (= q_1 , o estado inicial de M_0)

Passo 1 de simulação de M_0 :

1. U lê o símbolo sob o cabeçote da fita 2: é 0 (índice X_2).
2. U lê o estado atual da fita 3: é q_1 (representado como 0 , ou seja, 0^1).
3. U varre a fita 1 procurando a transição que começa com "\$ q_1 , X_2 \$", ou seja, com prefixo 0100 . Encontra na primeira (e única) transição.
4. U extrai a saída: (q_2, X_2, D_2) , ou seja, novo estado q_2 , escreve 0 , move à direita.
5. U atualiza:
 - Fita 2: escreve 0 na posição atual, move cabeçote para a direita. Fita 2 = 0 ___ com cabeçote em posição 2.
 - Fita 3: substitui o conteúdo por 00 (índice de q_2).

Verificação de aceitação. U sabe quais estados de M_0 são finais (está codificado na fita 1, embora nosso esquema simples não tenha guardado essa informação explicitamente; um esquema mais completo inclui a lista de finais). Como q_2 é final, U **aceita**.

UTM em Python

A MT Universal pode ser escrita em ~30 linhas de Python. Essencialmente, é um **laço** que lê instruções de uma tabela e as aplica a uma fita:

```
def utm(M, w):
    fita = list(w) + ['_'] * 1000          # fita simulada (com folga em branco)
    cabecote = 0
    estado = M['inicial']
    while estado not in M['finais']:
        simbolo = fita[cabecote]
        if (estado, simbolo) not in M['delta']:
            return False                    # rejeita por parada nao-final
        novo_estado, escrita, movimento = M['delta'][(estado, simbolo)]
        fita[cabecote] = escrita
        if movimento == 'R': cabecote += 1
        elif movimento == 'L': cabecote -= 1
        estado = novo_estado
    return True                            # aceita
```

Observe que esta função **simula qualquer MT** (qualquer dicionário **M**) sobre qualquer entrada **w**. Esse Python é, em essência, uma MT Universal.

Implicação. Cada interpretador de linguagem de programação (CPython, GCC, etc.) é, no fundo, uma MT Universal. O conceito que Turing definiu em 1936 é exatamente o mesmo que roda na sua máquina hoje.

💡 Por que a MT Universal é importante

A construção de U tem três consequências centrais:

1. **Toda MT pode ser programa.** A descrição $\langle M \rangle$ deixa de ser um conceito abstrato e vira **dado**. Você pode "carregar" qualquer MT em U e executá-la.

2. **O hardware pode ser fixo.** Não é preciso construir um circuito novo para cada algoritmo. Basta um único dispositivo (em última análise, U) e variar o **programa**. Esta é a base teórica da arquitetura **von Neumann** e de todo computador moderno.

3. **Permite raciocinar sobre MTs como objetos matemáticos.** Como $\langle M \rangle$ é uma string, podemos:

- comparar MTs (igualdade de strings);
- enumerar todas as MTs em ordem lexicográfica;
- construir MTs cuja entrada é a codificação de outra MT.

A última propriedade é fundamental. É ela que permite formular perguntas como "dada uma MT M , ela aceita a entrada w ?". Essa pergunta é o **Problema da Aceitação**, e ela é... **indecidível**.

5. A LINGUAGEM A_{TM} E A DIAGONALIZAÇÃO

Definindo A_{TM} (Problema da Aceitação)

A linguagem A_{TM} é o conjunto de pares $\langle M, w \rangle$ em que M aceita w :

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ é uma MT e } M \text{ aceita } w \}.$$

O que significa decidir A_{TM} ? Seria possuir uma MT H que, recebendo qualquer descrição $\langle M, w \rangle$, sempre para e devolve:

- **Aceita** se M aceita w ;
- **Rejeita** se M não aceita w (rejeita ou entra em loop).

Em termos de Python: H seria uma função `aceita(programa, entrada)` que devolve `True` ou `False`, **garantidamente** em tempo finito, para qualquer programa e qualquer entrada.

Por que isso seria útil? Resolveria de uma só vez um número astronômico de perguntas: "esse código vai produzir o resultado X?", "esse programa vai imprimir 'Hello, World'?", "esse algoritmo trata o caso da entrada vazia corretamente?". Tudo viraria A_{TM} com escolhas adequadas de M e w .

☰ Exemplos concretos de $\langle M, w \rangle$

Para fixar o que está em A_{TM} e o que não está, considere algumas MTs vistas (ou imagináveis):

MT M	Entrada w	M aceita w ?	$\langle M, w \rangle \in A_{TM}$?
M_{01} da Aula 7 (decide $\{0^n1^n\}$)	0011	sim	sim
M_{01}	001	não (rejeita)	não
M_{01}	ϵ	sim (caso $n = 0$)	sim
$M_{\#}$ (decide $\{w\#w\}$)	01#01	sim	sim
$M_{\#}$	01#10	não (rejeita)	não
M_{loop} que sempre roda em loop	qualquer	não (loop)	não
$M_{só\ aceita}$ que sempre aceita	qualquer	sim	sim

Observação. Para todos esses casos específicos, **é fácil** decidir se $\langle M, w \rangle$ está em A_{TM} : basta executar M sobre w e ver o que acontece (quando M é uma MT bem comportada que sempre para).

O drama: existem MTs para as quais **não dá** para saber se vão aceitar, rejeitar, ou rodar para sempre. E essas existem em quantidade infinita. É essa existência que torna A_{TM} indecidível no caso geral.

✓ A_{TM} é Recursivamente Enumerável

Teorema. $A_{TM} \in \mathbf{RE}$.

Prova (esboço). Basta construir uma MT V que reconhece A_{TM} :

1. Receba $\langle M, w \rangle$ na fita.
2. Decodifique M e w .
3. **Simule** M sobre w usando a MT Universal U .
4. Se M aceita w , então U atinge um estado final. Faça V aceitar.
5. Se M rejeita w , então U para sem aceitar. Faça V rejeitar.
6. Se M entra em loop sobre w , então U também entra em loop. V nunca para.

Note que V se comporta como esperado quando $\langle M, w \rangle \in A_{TM}$: ela aceita. Mas, quando $\langle M, w \rangle \notin A_{TM}$, V pode **não parar** (caso M entre em loop sobre w).

Isso é suficiente para mostrar $A_{TM} \in \mathbf{RE}$. Resta saber: V pode ser melhorada para **sempre parar**? Ou seja, A_{TM} é Recursiva?

✗ A_{TM} não é Recursiva (Turing, 1936)

Teorema. $A_{TM} \notin \mathbf{R}$.

Equivalentemente: **não existe** algoritmo que decida, para todo par $\langle M, w \rangle$, se M aceita w .

Estratégia da prova: **contradição por diagonalização**. Suponha, por absurdo, que existe uma MT H que decide A_{TM} :

$$H(\langle M, w \rangle) = \begin{cases} \text{aceita} & \text{se } M \text{ aceita } w; \\ \text{rejeita} & \text{caso contrário.} \end{cases}$$

A MT H **sempre para**. Vamos derivar uma contradição a partir da existência de H .

A peça-chave. Usando H , construímos uma **nova MT** D com comportamento "contrário" e, ao perguntar a D sobre si mesma, chegamos em uma impossibilidade lógica. O próximo slide detalha a construção.

↻ A MT diabólica D

Construímos uma MT D que, dada a descrição $\langle M \rangle$ de uma MT M , decide se M aceita ou não a sua própria descrição, e faz o contrário.

Pseudocódigo de D , sobre entrada $\langle M \rangle$:

1. Chame H sobre $\langle M, \langle M \rangle \rangle$ (pergunte: "\$M\$ aceita $\langle M \rangle$?").
2. Se H responde "aceita", então D **rejeita**.
3. Se H responde "rejeita", então D **aceita**.

Em uma linha:

$$D(\langle M \rangle) = \begin{cases} \text{rejeita} & \text{se } M \text{ aceita } \langle M \rangle; \\ \text{aceita} & \text{se } M \text{ não aceita } \langle M \rangle. \end{cases}$$

Observação. D é uma MT bem definida porque H é uma MT (por hipótese) e H sempre para. Portanto D também sempre para.

🔴 O golpe final: rode D sobre $\langle D \rangle$

Como D é uma MT, ela tem uma descrição $\langle D \rangle$. Pergunta inevitável: o que acontece quando rodamos D sobre a entrada $\langle D \rangle$?

Aplicando a definição de D com $M = D$:

$$D(\langle D \rangle) = \begin{cases} \text{rejeita} & \text{se } D \text{ aceita } \langle D \rangle; \\ \text{aceita} & \text{se } D \text{ não aceita } \langle D \rangle. \end{cases}$$

Analise os dois casos:

- **Caso 1:** se $D(\langle D \rangle)$ aceita, então (pela definição) D deveria rejeitar $\langle D \rangle$. Contradição.
- **Caso 2:** se $D(\langle D \rangle)$ rejeita, então (pela definição) D deveria aceitar $\langle D \rangle$. Contradição.

Em qualquer caso, contradição. A única hipótese que podemos descartar é a existência de H .

Conclusão. H não existe. Não há MT que decida A_{TM} . Portanto $A_{TM} \notin \mathbf{R}$.

🕒 Por que se chama "diagonalização"?

A técnica usada por Turing tem o mesmo espírito da **diagonalização de Cantor** (que prova que os reais são incontáveis).

Imagine uma tabela infinita com:

- **Linhas:** todas as MTs $M_1, M_2, M_3, \dots$ (enumeradas pelas suas descrições).
- **Colunas:** todas as strings de entrada $w_1, w_2, w_3, \dots$ (em ordem lexicográfica).
- **Célula (i, j) :** "aceita" se M_i aceita w_j ; "rejeita" caso contrário.

	w_1	w_2	w_3	w_4	$\dots$
M_1	A	R	A	R	$\dots$
M_2	A	R	A	A	$\dots$
M_3	A	A	A	R	$\dots$
M_4	R	A	R	R	$\dots$
$\vdots$					$\ddots$

A MT D é construída para **diferir da diagonal**: na entrada $w_i = \langle M_i \rangle$, D devolve o **oposto** do que M_i devolve.

Conclusão visual. D não pode estar na tabela: ela difere de M_i na i -ésima posição. Mas se D existe como MT, ela **deveria estar** na tabela. Contradição.

6. O PROBLEMA DA PARADA

Definindo HALT_{TM}

A linguagem do **Problema da Parada** é

$$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ para sobre a entrada } w \}.$$

Pertencer a HALT_{TM} significa: ao rodar M sobre w , depois de algum número finito de passos, M **termina** (aceitando ou rejeitando). Não importa qual delas; importa apenas que pare.

Pergunta natural. Existe um algoritmo que, dado um programa M e uma entrada w , decide se M termina sobre w ?

Em termos de Python: existe `para(programa, entrada)` que sempre devolve `True` ou `False` corretamente?

Teorema (Turing, 1936). HALT_{TM} é **indecidível**. Ou seja, $\text{HALT}_{\text{TM}} \notin \mathbf{R}$.

Esse é o resultado mais famoso da Teoria da Computação. É o equivalente do Teorema da Incompletude de Gödel para o universo dos programas.

🔗 HALT_{TM} é equivalente a A_{TM}

Teorema. Se HALT_{TM} fosse decidível, então A_{TM} também seria.

Prova (esboço). Suponha que existe uma MT K que decide HALT_{TM} . Construimos uma MT H que decide A_{TM} :

Sobre entrada $\langle M, w \rangle$:

1. Pergunte a K : "\$M\$ para sobre w ".
2. Se K diz **não**, então M entra em loop, logo **não aceita** w . H rejeita.
3. Se K diz **sim**, então M para sobre w . Simule M sobre w (com U) até M parar. Aceite ou rejeite conforme M tenha aceitado ou rejeitado.

Observe: H sempre para, porque (1) K sempre para e (2) na linha 3, sabemos que M vai parar, então a simulação termina em tempo finito. Logo H decide A_{TM} .

■ **Mas** já sabemos que A_{TM} é indecidível! Portanto K não pode existir. **Conclusão:** HALT_{TM} é indecidível.

O Problema da Parada em Python

A indecidibilidade de HALT_{TM} tem uma versão informal em Python. Suponha que existisse `para(p, e)` que devolve `True` se o programa `p` termina sobre a entrada `e`. Definimos:

```
def estranho(p):
    if para(p, p):
        while True:
            pass          # entra em loop
    else:
        return           # termina
```

O que acontece se chamarmos `estranho(estranho)`?

- Se `para(estranho, estranho)` devolve `True`, então `estranho(estranho)` entra em loop, ou seja, **não para**. Mas `para` afirmou que ele para. Contradição.
- Se `para(estranho, estranho)` devolve `False`, então `estranho(estranho)` retorna, ou seja, **para**. Mas `para` afirmou que ele não para. Contradição.

Em palavras simples: se você tem uma função `para`, eu uso `para` para programar uma função que se nega a si mesma. Essa negação é uma contradição lógica, então a função `para não pode existir`. Não importa qual linguagem você escolha.

⚠ Consequências práticas da indecidibilidade

Como HALT_{TM} é indecidível, **nenhum** programa consegue, no caso geral, responder se outro programa termina. Consequências:

- **Verificadores de código nunca são completos.** Toda ferramenta de análise estática (Pylint, Coverity, SonarQube) pode dar **falsos positivos** ou **falsos negativos**. Não é deficiência de engenharia; é uma barreira lógica.
- **Compiladores não podem decidir todas as otimizações.** "Esse loop sempre termina?", "esse trecho é código morto?" são perguntas indecidíveis no caso geral.
- **Verificação formal exige restrições.** Sistemas como Coq, Isabelle e Agda evitam o problema **restringindo** o que é programável (linguagens totais, sem recursão geral).
- **Não existe um "antivírus perfeito".** Pelo Teorema de Rice (Aula 9), perguntas semânticas sobre programas são indecidíveis em geral. Detectar malware perfeitamente é impossível.

Mensagem fundamental. A indecidibilidade não é uma limitação da MT; é uma propriedade **da computação**. Ela ressurge em qualquer formalismo equivalente. Engenheiros de software lidam com essa barreira **todos os dias**, ainda que não conheçam o nome.

7. LINGUAGENS FORA DE RE

≠ Complemento de uma linguagem

Dada uma linguagem $L \subseteq \Sigma^*$, seu **complemento** é

$$\overline{L} = \Sigma^* \setminus L = \{w \in \Sigma^* \mid w \notin L\}.$$

Observação importante. Linguagens em **R** são **fechadas sob complemento**:

Se L é decidida por uma MT M que sempre para, basta trocar os papéis de "aceita" e "rejeita" em M para obter M' que decide $\overline{L}$. Logo $\overline{L} \in \mathbf{R}$.

E em RE? A classe **RE não** é fechada sob complemento. Isto é, existe $L \in \mathbf{RE}$ tal que $\overline{L} \notin \mathbf{RE}$.

A demonstração desse fato é o que nos permite **subir** mais um andar na hierarquia: identificar linguagens que **nenhuma** MT reconhece.

Lema da dupla reconhecibilidade

Lema. Se $L \in \mathbf{RE}$ e $\bar{L} \in \mathbf{RE}$, então $L \in \mathbf{R}$.

Prova (esboço). Sejam M_L reconhecendo L e $M_{\bar{L}}$ reconhecendo $\bar{L}$. Construimos D que decide L :

Sobre entrada w :

1. Simule, **em paralelo** (em fitas separadas, ou intercalando passos), M_L e $M_{\bar{L}}$ sobre w .
2. Como w está em L ou em $\bar{L}$, **exatamente uma** das duas vai parar e aceitar.
3. Se M_L aceita primeiro, D aceita. Se $M_{\bar{L}}$ aceita primeiro, D rejeita.

D sempre para porque sempre uma das duas MTs aceita em tempo finito. Logo D decide L , portanto $L \in \mathbf{R}$.

Em palavras simples: se você reconhece tanto L quanto seu complemento, você decide L . Basta correr os dois reconhecedores em paralelo.

⊗ $\overline{A_{TM}}$ não é RE

Teorema. $\overline{A_{TM}} \notin \mathbf{RE}$.

Prova. Suponha, por absurdo, que $\overline{A_{TM}} \in \mathbf{RE}$.

- Já provamos que $A_{TM} \in \mathbf{RE}$.
- Pelo lema da dupla reconhecibilidade, segue que $A_{TM} \in \mathbf{R}$.
- Mas isso contradiz o teorema da diagonalização: $A_{TM} \notin \mathbf{R}$.

Logo, $\overline{A_{TM}}$ **não pode** estar em $\mathbf{RE}$.

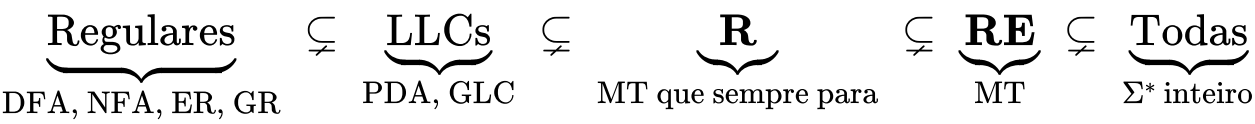
Em palavras: existem strings em $\{0, 1\}^*$ (especificamente, descrições $\langle M, w \rangle$ onde M **não** aceita w) que **nenhuma** MT consegue reconhecer como tais. Não há nenhum algoritmo, nem mesmo um que possa entrar em loop, capaz de identificar com certeza esses pares.

Implicação. O universo das linguagens em Σ^* tem **três camadas**: as decidíveis (R), as apenas reconhecíveis (RE - R), e as **inatingíveis** (fora de RE). A hierarquia que tínhamos até a Aula 7 fica completa agora.

8. HIERARQUIA FINAL

 A hierarquia completa

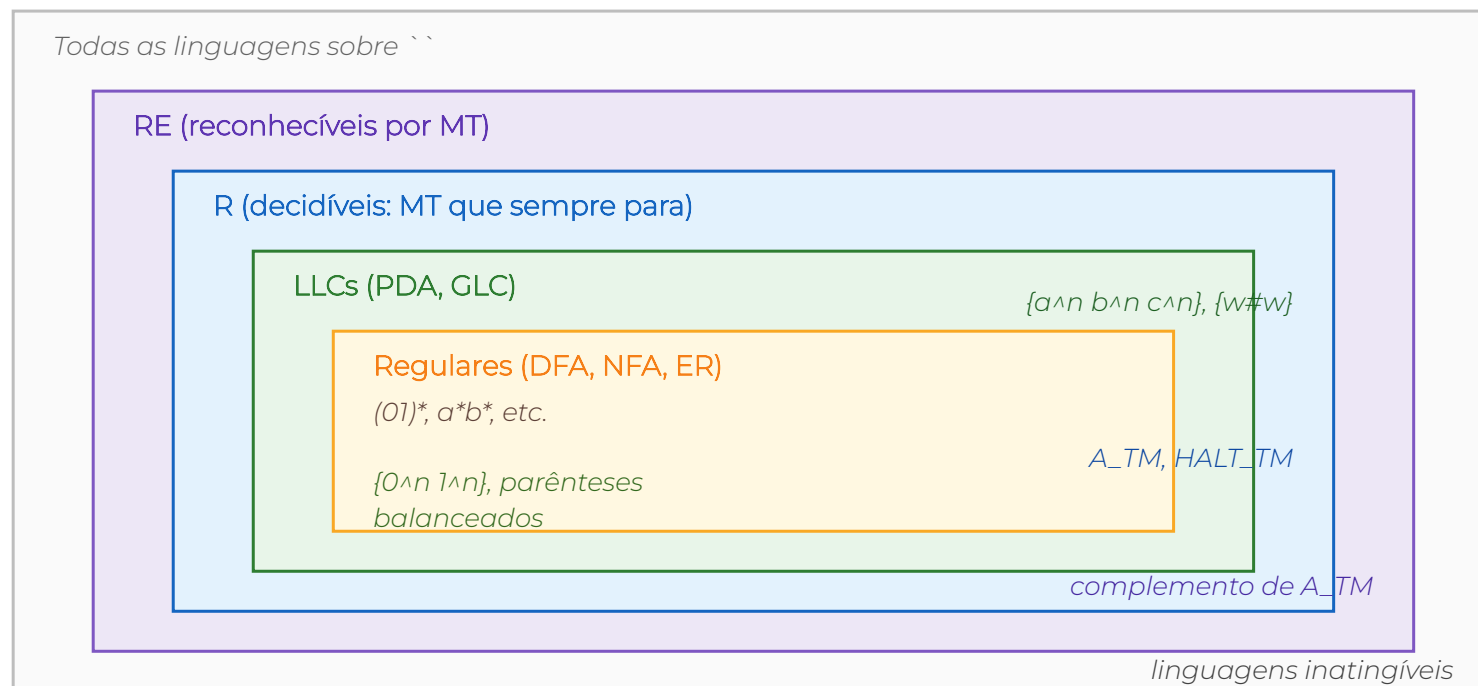
Reunindo tudo o que vimos da Aula 2 até aqui:



Classe	Modelo	Exemplo dentro	Exemplo fora
Regulares	DFA, NFA	$(01)^*$	$\{0^n 1^n\}$
LLCs	PDA, GLC	$\{0^n 1^n\}$	$\{a^n b^n c^n\}$
R (Recursivas)	MT que sempre para	$\{a^n b^n c^n\}, \{w \# w\}$	$A_{\text{TM}}, \text{HALT}_{\text{TM}}$
RE (Rec. Enumeráveis)	MT	$A_{\text{TM}}, \text{HALT}_{\text{TM}}$	$\overline{A_{\text{TM}}}$
Todas as linguagens	(nenhum modelo)	$\overline{A_{\text{TM}}}$	(não há "fora")

Mensagem central. Existem mais linguagens do que MTs (cardinalidade): linguagens são subconjuntos arbitrários de Σ^* , e o conjunto das MTs é enumerável. Logo, a maioria das linguagens **não** está em **RE**. As "linguagens computáveis" são uma minoria estatística.

Visualização da hierarquia



Cada anel é um superconjunto **estrito** do anterior. Exemplos no contorno indicam linguagens que estão **no anel** mas **fora** do anel imediatamente interior.

🔧 Por que isso importa?

Os resultados desta aula não são apenas teoria abstrata. Eles delimitam o que a Ciência da Computação **pode** e **não pode** fazer:

- **Engenharia de software.** Toda ferramenta de análise de código (debuggers, profilers, linters, fuzzers) opera dentro de classes decidíveis, geralmente restringindo o tipo de programa analisado.
- **Compiladores.** Otimizações como eliminação de código morto, análise de fluxo, predição de saltos enfrentam barreiras de decidibilidade. Soluções práticas usam **aproximações conservadoras**.
- **Segurança.** Detecção de malware é, no caso geral, indecidível (Teorema de Rice). Ferramentas reais usam heurísticas, assinaturas e técnicas de aprendizado de máquina, sem garantias formais de cobertura.
- **Matemática automatizada.** Provadores de teoremas (Coq, Lean, Isabelle) operam sobre fragmentos decidíveis da lógica. O fragmento totalmente expressivo (lógica de primeira ordem completa) é indecidível.
- **Computação quântica.** Mesmo computadores quânticos **não** escapam da indecidibilidade. Eles aceleram problemas dentro de **$\mathbf{R}$** , mas não resolvem **A_{TM}** .

Em uma frase: os resultados de 1936 de Turing e Church definem o **horizonte da computação**. Tudo o que veio depois opera dentro desses limites.

9. EXERCÍCIOS

✓ Questões objetivas

1. Por que a Tese de Church-Turing é classificada como tese, e não como teorema? Responda em uma frase.

■ Dica. Pense no que ela conecta: um conceito **informal** e um conceito **formal**.

2. Verdadeiro ou falso? *Toda MT multi-fita é mais poderosa que uma MT de fita única.* Justifique.

■ Dica. Lembre do Teorema 5.4.1: o que muda entre multi-fita e fita única?

3. Verdadeiro ou falso? *Toda string $\langle x \in \{0,1\}^* \rangle$ pode ser interpretada como a codificação de uma MT.** Justifique.

■ Dica. Lembre da convenção da MT padrão $M_{\perp}$.

4. Dada a relação $A_{TM} \in \mathbf{RE}$, o que podemos afirmar sobre $\overline{A_{TM}}$? Marque a alternativa correta:

- (a) $\overline{A_{TM}} \in \mathbf{R}$
- (b) $\overline{A_{TM}} \in \mathbf{RE}$ mas talvez não em $\mathbf{R}$
- (c) $\overline{A_{TM}} \notin \mathbf{RE}$
- (d) Nada se pode afirmar

■ Dica. Use o lema da dupla reconhecibilidade junto com o fato de que $A_{TM} \notin \mathbf{R}$.

5. Verdadeiro ou falso? *Existe um programa Python que, dado o código fonte de outro programa Python e uma entrada, decide se aquele programa termina sobre aquela entrada.* Justifique.

■ Dica. Pense na Tese de Church-Turing e no Problema da Parada.

❓ Questões discursivas

6. Considere uma MT M qualquer com dois estados finais e três símbolos da fita. Quantos bits, aproximadamente, ocupa a codificação $\langle M \rangle$ pelo esquema descrito na Seção 3? Justifique.

| Dica. Conte transições possíveis e multiplique pelo tamanho médio de cada transição em codificação unária.

7. Mostre que a classe $\mathbf{R}$ é fechada sob **união**. Dadas $L_1, L_2 \in \mathbf{R}$, descreva uma MT M que decide $L_1 \cup L_2$ e argumente que ela sempre para.

| Dica. Pense em rodar as duas MTs em sequência e devolver "aceita" se qualquer uma aceitar.

8. Mostre que a classe $\mathbf{RE}$ é fechada sob união. Cuidado: como uma das MTs pode entrar em loop, o esquema do exercício 7 **não funciona** diretamente. Adapte usando simulação em paralelo.

| Dica. Use a mesma ideia do lema da dupla reconheçibilidade: alterne passos das duas MTs.

9. Considere a linguagem $L_\varepsilon = \{\langle M \rangle \mid M \text{ aceita a string vazia } \varepsilon\}$. Argumente, sem prova formal, por que L_ε é indecidível.

| Dica. Tente reduzir A_{TM} a L_ε : dado $\langle M, w \rangle$, construa uma MT M' tal que M' aceita ε se e somente se M aceita w .

10. Em um ensaio curto (10 a 15 linhas), explique a um colega de outro curso (sem usar a palavra "Turing") por que **não pode existir um programa que decide se outro programa entra em loop**. Termine sua resposta com uma consequência prática para a engenharia de software.

| Dica. Construa a versão informal: "se existisse, eu o usaria para fazer um programa que se contradiz".

🚩 Encerramento da Aula 8

O que vimos:

- **Tese de Church-Turing**: equivalência entre MT e qualquer modelo razoável de computação.
- **Variantes da MT** (multi-fita, não determinística) são equivalentes em poder à MT padrão.
- **Codificação** $\langle M \rangle$: toda MT vira string em $\{0, 1\}^*$.
- **MT Universal** U : uma MT que executa qualquer outra. Base teórica do computador programável.
- A_{TM} é RE mas não é Recursiva, demonstrado por **diagonalização**.
- **Problema da Parada** ($HALT_{TM}$) é indecidível, com versão em Python.
- A_{TM} não é RE: existem linguagens **inatingíveis**.
- Hierarquia final: Regulares $\subsetneq$ LLCs $\subsetneq$ **R** $\subsetneq$ **RE** $\subsetneq$ Todas.

Próxima aula (Análise Léxica): aplicação dos DFA/NFA na construção do **scanner** de um compilador; especificação de tokens via ER; geração de tabelas de transição; ferramentas Flex/Lex.

Obrigado!

Dúvidas?